

Software for the Analysis of Atomic Spectra (SAAS)

Background:

Spectra from the Fourier transform (FT) and grating spectrometers at ICL and the National Institute of Standards and Technology (NIST, USA) and are currently analysed with a diverse set of programs written in languages from 1970's FORTRAN to python3. Many of these programs are hand-crafted by the individual researcher to match their needs, but this makes it difficult to get new students up and running rapidly, and hampers people from taking over an existing project to complete or extend it. In addition, each researcher has their own conventions for file names, variable names and short-hand spectroscopic notation that can be hard for other researchers to understand. Most of the analysed data are ASCII files, sometimes with different versions in different directories.

This document outlines the requirements and decisions needed to replace this diverse set of programs with a more homogeneous set of programs and data, with the goal of making it easier for new people to begin a project or take over an existing one. In addition, the new software should encode as much as possible of the existing knowledge of senior researchers, particularly with the evaluation of uncertainties and in line identification techniques.

Basic analysis principles:

Atoms can be excited in a discharge (e.g. the Sun, or a fluorescent light) and will then emit light in a characteristic pattern of wavelengths – a spectrum - that are unique to each atom. Each line in the spectrum results from an electron in the atom moving between two energy levels – a transition - emitting a photon whose energy depends on the difference of the two levels. The atoms can also be ionised – have an electron stripped off – and the ions produced have their own spectrum. In atomic spectroscopy, we measure these spectra and use them to obtain fundamental properties of the atom, such as the energy level structure and the Einstein A coefficients, also known as transition probabilities (actually transition rates, in number of photons emitted per second).

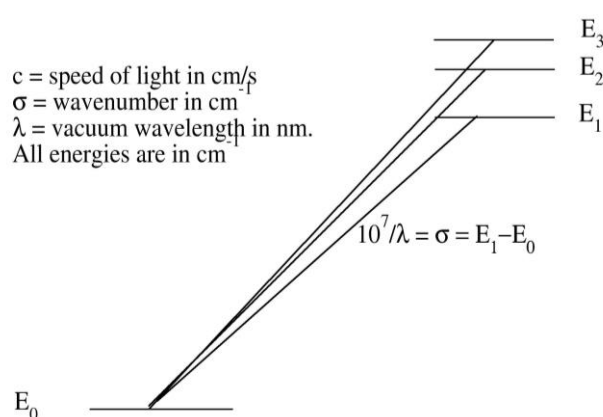


Figure 1: Three spectral lines emitted by electrons moving between energy levels

Figure 1 is a schematic diagram of the energy levels of an atom, consisting of four levels, E_0 , E_1 , E_2 and E_3 . These energy levels have various properties associated with them, including the energy,

E; the total angular momentum, J; and the parity (odd or even). In Fig. 1, the levels E_1 , E_2 and E_3 have the opposite parity to E_0 . For ‘allowed’ transitions, electrons can only move between levels of different parity (from odd to even or vice-versa), and the J-value can only change by 0, -1, or +1, with 0 – 0 transitions not allowed. These are called the selection rules. Photons from allowed transitions are usually the only ones seen in our spectra.

The energy of the emitted photon is the difference of the two energy levels ($E_1 - E_0$ in Fig. 1). In our measurements, the energy level values are given in cm^{-1} , with $1 \text{ cm}^{-1} \approx 2 \times 10^{-23} \text{ J}$. Our instruments measure either wavenumbers, σ , (number of waves per cm), or vacuum wavelengths in nm, λ . In older papers, wavelengths may be given in Ångströms, Å, where $1 \text{ Å} = 0.1 \text{ nm}$. Using these units, the wavenumbers are simply from the difference of the energy levels, and the vacuum wavelengths in nm are $10^7 / \lambda$. In our analysis, we derive the energy levels from a set of measured wavenumbers using a matrix inversion in an external program written in Perl and JavaScript. This is an iterative process, because problems with the measurements or the assignments of transitions to observed spectral lines can affect all of the energy levels – this is the most time-intensive part of the analysis technique. One complication is that wavelengths are sometimes given in air, λ_{air} , rather than vacuum with

$$\lambda_{\text{air}} = 10^7 / (\sigma (1 + 8342.13 \times 10^{-8} + 15997 / (3.89 \times 10^9 - \sigma * \sigma)) + 2406030 (130 \times 10^8 - \sigma * \sigma))$$

Another fundamental property that atomic spectroscopists measure is the transition probability A_{ul} (actual a rate, rather than a probability), which is the number of transitions that an electron will make from an higher energy level u to a lower energy level l in one second. Another term for these are Einstein A-coefficients, or simply A values. These transition probabilities, measured in the laboratory, are used to determine elemental abundances in astronomical spectra. We use the branching fraction (BF) method to determine experimental transition probabilities. The BF_{ul} for a transition from level u to level l is defined as:

$$BF_{ul} = \frac{A_{ul}}{\sum_{k=1} A_{uk}} \quad (1)$$

where the sum is over all of the lower levels that an electron in level u can decay to using the selection rules for allowed transitions described above. We determine these from the intensities I_{ul} of the spectral lines, in photons/s (i.e. a photon flux rather than an actual intensity in W). To obtain these intensities, our spectral lines need to be calibrated using a spectrum of a radiometric standard lamp taken at the same time as our spectrum of interest. Equation (1) can then be rewritten as:

$$BF_{ul} = \frac{I_{ul}}{\sum_{k=1} I_{uk}} \quad (2)$$

where I_{uk} is the calibrated intensity, and the sum is again over all spectral lines that are emitted from the upper level u. Finally, since the lifetime of the upper level τ_u is related to the transition probabilities of all transitions from that upper level by:

$$\sum_{k=1} A_{uk} = \frac{1}{\tau_u}$$

(3)

the desired transition probability can be obtained from:

$$A_{ul} = \frac{BF_{ul}}{\tau_u}$$

(4)

The transition probability can thus be derived if the lifetime of the upper level is known, and if all lines from the upper level can be measured. The last requirement is rarely possible since all lines will contribute to the sum, but not all lines are strong enough to be measured in the laboratory. The contribution from lines not measured is called the “residual”, which can be estimated using theoretical calculations.

In addition to the energy levels and transition probabilities we also sometimes measure the hyperfine and isotope structure of the energy levels. These come from the fact that the nucleus of an atom has a finite size, and the mass varies between the different isotopes, affecting the atomic energy levels. These properties are not the focus of our research at the moment but may well be at in the future.

Overview:

There are four main analysis areas that will be included in the program by the end of development. This structure is outlined in Figure 2: Overview of data inputs, analysis, and outputs. together with the data inputs and outputs of the program These are:

1. Term analysis: spectral wavelengths are used to determine the atomic structure of the atom/ion measured in the spectra. Outputs will be energy levels and identified wavelengths.
2. Transition probabilities: relative intensities of selected lines from an upper energy level are used to calculate branching fractions (fraction for a line of the total intensity of all lines from that upper energy level). The BFs are combined with the lifetime of the upper energy level to produce transition probabilities i.e. how likely each transition is to occur.
3. Hyperfine structure: interaction between the nuclear moment and orbiting electrons causes fine structure energy levels to split, which then splits spectral lines into multiple components. Analysis of these spectral line profiles gives hyperfine constants for the upper and lower energy levels.
4. Isotope structure: different isotopes of an atom/ion can shift energy levels, splitting spectral lines into multiple components. Fitting of profiles to these lines produces the exact wavelengths of the component lines.

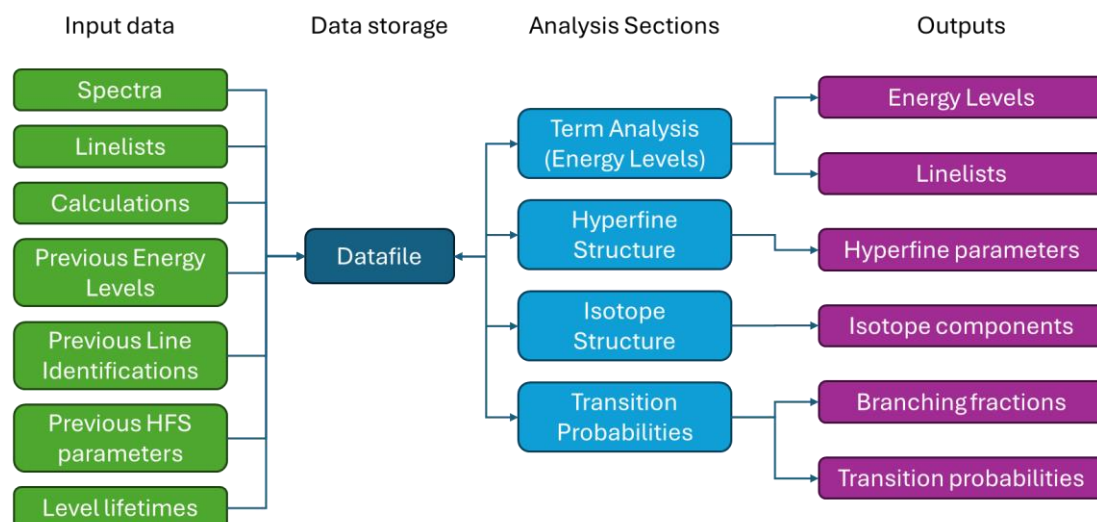


Figure 2: Overview of data inputs, analysis, and outputs.

Initially, this project would focus on the transition probability analysis. This would then give us the framework and knowledge of data handling, storage, GUI elements etc. to carry on the work and extend to the other analysis areas or for RSE to continue the work if funding/time permits.

The main objectives would be to get:

- Expertise and experience:
 - We do not have experience with designing larger software with GUIs and data handling and flow.
- Framework:
 - Initially, one fully working analysis “area” (this would be transition probabilities)
 - main datafile design.
 - GUI
 - Data flow and handling
 - I/O for input files and conversion into main datafile
 - Plotting of spectra, filtered curves etc.

The key requirements of the software are:

- Ideally written in python (group experience + likely student language)
- One piece of software to be used by all researchers (across several research groups)
 - Easy to collaborate on further development (Github)
 - Maintainable code
 - Able to write documentation for student researchers.
- One datafile for all inputs and outputs (but open to other advice)
 - All data in one easy to find location.
 - Not multiple files spread across multiple locations. We want all data related to the element/ionisation in one place for easy access, viewing and analysis.
 - Easy to transfer all needed data.
 - This would be between researchers and to continue previous analyses.

- There are not likely to be large numbers of researchers working on the same project simultaneously. Two users are likely the maximum number.
- The key requirements of the data storage solution are:
 - **Viewable:** we would like to see exactly what data is present for the element and what it contains. This would mean being able to view input/output datafiles from within the program.
 - **Version-able:** to be able to track the analysis. This will be useful for future researchers as guides or if another researcher takes over the analysis.
- Knowledge (techniques, pitfalls, uncertainties etc) embedded into code.
- Be able to link results from one analysis back into future/past analyses e.g. HFS components into energy level analyses.
- A wizard for importing data into the analysis. This would enable sanity checking of the data and allow the user to specify parameters about the data (e.g. calibration certificate global uncertainty, band limits of spectra etc.)
- A wizard/option dialog for determining data outputs from the analysis.

Data Storage

One of the main aspects we are keen to have advice on is the data storage aspect of the program. We would like to be able to store all data related to an analysis in a logical, easy to access and view format. We have been exploring the HDF5 format and it appears to have many possible advantages for this project. We are interested to hear the opinion of the RSE team as to whether this is appropriate for the project or whether there are other formats that would suit our needs better. Although the amount of data is not large by current standards, a typical analysis may contain 15 – 20 spectra, each up to 32 MB in size. When combined with associated inputs (calculated parameters, previously published data, previous analyses, and data for other ionisation stages), the task of analysing the spectra becomes problematic if they are stored as flat files. It also poses a problem when transferring a project to students and other researchers, as important files are sometimes omitted, and a spectrum that may be of value to the student can be overlooked. By having all these data in one place we will be able to extract the most science from our experimental data e.g. energy level analysis spectra can also be used to analyse hyperfine structure components etc.

Main program outline for transition probability analysis

This section outlines the broad analysis techniques for transition probabilities, gives details of the input files needed for analyses and provides details for the steps needed in the analysis of transition probabilities.

Data inputs

The data inputs to the analysis of transition probabilities will be multiple spectra of the element (referred to as main spectra), each with associated standard lamp spectra or response functions. Each main spectra will have at least one associated linelist (list of all spectral lines of interest in the spectrum with wavenumber, intensity, S/N etc.). Each main spectrum will likely have:

- Interferogram files (.hdr & .dat) – these are the raw experimental data
- Transformed spectrum files (.hdr & .dat) – interferograms are Fourier transformed to spectra using the xgremlin program
- Associated transform files (.badpt, .phase etc) – data about the Fourier transform process from xgremlin. For reference only – not used in analysis.
- Linelists (potentially multiple in .lin or .lst format) – Data for fitted lines in the spectrum. There may be multiple linelists for a spectrum.
- Response function for intensity calibration - enables calibration of line intensities.
- Standard lamp spectra for intensity calibration (multiple .hdr and .dat files)

The analysis of the spectral lines in these spectra is the bulk of the transition probability analysis. In addition, there will be energy level values and lifetimes, calculated transition probabilities, previously measured line data and calibration certificate data used in the analysis.

The following data files are used for the start of the analysis of FTS data for transition probabilities. Some of these data inputs are outputs of another piece of software (xgremlin) and so we would prefer that that file format be kept the same to minimise user work. However, there are many other data inputs that would need to be compiled by the user and so we are free to specify any file format we like.

This is a detailed list of the input files needed for transition probability analyses and their formats.

1. Element under study spectrum
 - a. These will either be NSO or FITS filetypes:
 - i. NSO:
 1. Includes both a .hdr and .dat file (header and data files)
 2. hdr files are ASCII.
 3. . dat files are 32-bit floating point numbers.
 - ii. FITS:
 1. Standard FITS file format
2. Linelist file
 - a. Contains parameters for fitted lines of a spectrum.
 - b. Must be an xgremlin .lin or .aln:
 - i. .lin is a binary format file
 - ii. .aln is an ascii linelist file from xgremlin “writelines” command output
3. Levels file
 - a. Contains all known levels for the species under study from literature.
 - b. CSV
 - c. Header containing meta data about the levels? E.g. Source, date accessed etc.
 - d. Columns are:
 - i. desig – energy level label (datatype: utf-8 string)
 - ii. J – J value of the level (datatype: utf-8 string)
 - iii. energy – energy of level in cm-1 (datatype: float)
 - iv. energy uncertainty – uncertainty of level energy in cm-1 (datatype: float)
 - v. parity – parity of level (0 for even or 1 for odd) (datatype: utf-8 string)
 - vi. species – name of element and ionisation stage (datatype: utf-8 string)
 - vii. lifetime – lifetime of the level in ns (datatype: float)
 - viii. lifetime uncertainty – uncertainty of level lifetime in ns (datatype: float)
4. Previous Identifications file (optional):

- a. Contains all previously measured lines, but only one instance of each line if combining multiple sources (e.g. FTS and grating)
 - b. CSV
 - c. Header to contain metadata about file source
 - d. All level labels MUST match with the “desig” specified in the Levels file
 - e. Columns are:
 - i. wavenumber – wavenumber of line in cm⁻¹ (datatype: float)
 - ii. lower energy – lower level energy in cm⁻¹ (datatype: float)
 - iii. upper energy – upper level energy in cm⁻¹ (datatype: float)
 - iv. lower desig – lower level label (datatype: utf-8 string)
 - v. upper desig – upper level label (datatype: utf-8 string)
 - vi. snr – signal to noise ratio of line (datatype: float)
 - vii. intensity – calibrated relative intensity of line (datatype: float)
 - viii. notes – any comments/notes about the line (datatype: utf-8 string)
5. Calculations file:
- a. Contains calculations for transition probabilities of species’ lines.
 - b. CSV
 - c. Header with all info about source (Method of calculation, who calculated etc.)
 - d. All level labels MUST match with “desig” specified in the Levels file
 - e. Columns are:
 - i. lower desig – upper energy level label (datatype: utf-8 string)
 - ii. upper desig – lower energy level label (datatype: utf-8 string)
 - iii. log(gf) – log(gf) value of line (datatype: float)
6. Calibration Certificate Data:
- a. Contains calibrated radiances of standard lamps for intensity calibration
 - b. CSV
 - c. Header with source information (calibration location, date etc.)
 - d. Columns are:
 - i. Wavelength (nm)
 - ii. Absolute spectral radiance (Wcm⁻²sr⁻¹nm⁻¹)
 - iii. Absolute spectral radiance (mWm⁻²sr⁻¹nm⁻¹) - optional
 - iv. Uncertainty (%) (if missing from import, user will need to specify a global uncertainty for the certificate data)
7. Response function (optional):
- a. Gives total response R at given wavenumber. This will calibrate intensities of spectral lines onto a relative scale.
 - b. Each spectrum will either have the response function file, or standard lamp spectra with associated calibration certificate data.
 - c. Columns are:
 - i. wavenumber (cm⁻¹)
 - ii. response

Main transition probability analysis schema

An outline of the main steps in the analysis of transition probabilities is shown in Figure 3. The blue boxes are the main steps, with details of each step given in orange. The green boxes are for the optional creation of the response function if one is not supplied by the user.

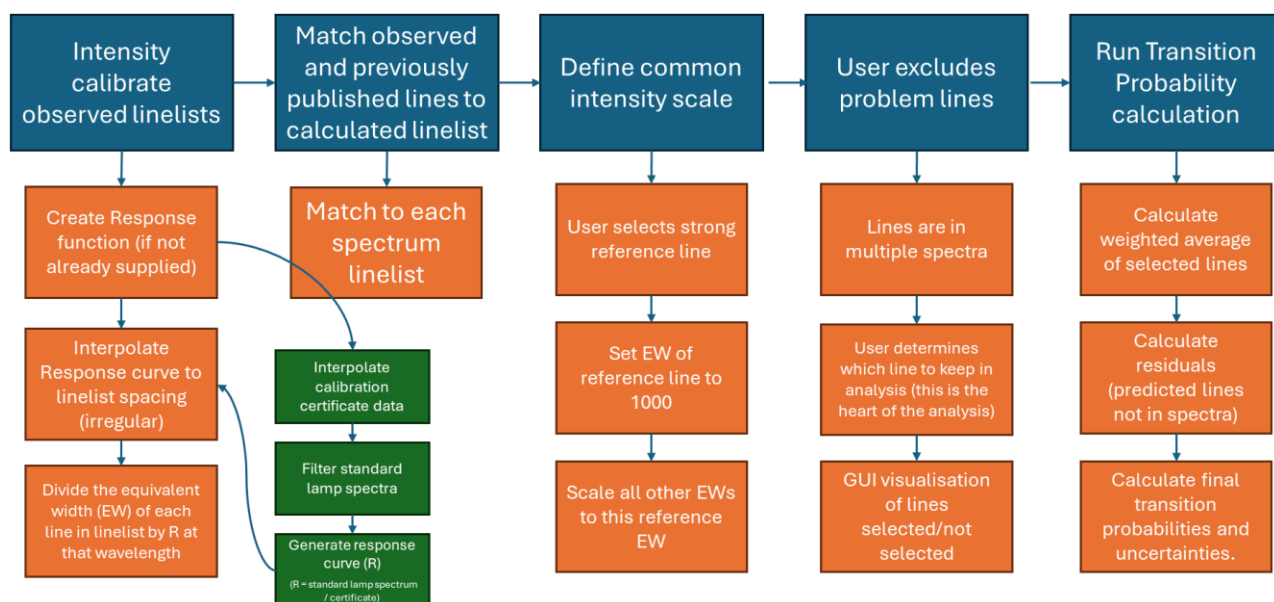


Figure 3: Basic flowchart of the main steps in the Transition Probability analysis

Details of the analysis steps are detailed below.

1. Intensity calibration of observed linelists:

Each observed linelist will need to be intensity calibrated to place all spectral line intensities on the same relative scale.

Each main element spectrum will either have:

- A response function, calculated externally by the user, or
- Standard lamp spectra and calibration certificate data, from which we can calculate a response function. There are likely to be two standard lamp spectra for each main element spectrum, one taken before the main spectrum and one after to ensure that the measurement conditions (and therefore the instrument response) did not change drastically during the main measurement period.

If no response function is supplied by the user, then the steps for calculating a response function are:

1. Filter the before and after standard lamp spectra:
 - a. First, filter the spectrum to match resolution of the spectrometer used for the calibration. This step is not necessary if the spectrum is a pure continuum.
 - i. Interpolate the standard lamp spectrum onto a wavelength scale, keeping the number of points similar.
 - ii. Filter the spectrum with two boxcars, corresponding to the input and exit slit of the spectrometer used for the calibration. For PTB, this is roughly 0.92 nm and 0.46 nm, determined from the profile of Lyman-alpha.
 - iii. Interpolate the spectrum back onto the original wavenumber scale.
 - b. Open a new window with the standard lamp spectrum and filter options.

- i. We would like a range of filters here, to reduce noise in the spectrum with parameters chosen by the user.
 - ii. plot the original spectrum with the filtered spectrum overlayed
 - iii. when user happy, save the filtered spectrum as a new dataset.
 - c. Plot the before and after filtered standard lamp spectra and their average filtered spectrum in a new window.
 - i. get the user to select which spectrum will be used for the intensity calibration (default: average). This step is in case there is an issue with one of the standard lamp spectra and it can therefore be discarded.
- 2. Interpolate the calibration certificate data:
 - a. interpolate the calibration certificate to the number of data points in the filtered standard lamp spectrum.
 - b. this needs to be in a photons per wavenumber scale:
 - i. original certificate data will be in $\text{W per cm}^{-2} \text{ sr}^{-1} \text{ nm}^{-1}$
 - 1. so will need to convert certificate data.
 - 2. this conversion is $\text{original data} * \text{wavelength}^3$.
 - ii. save this as a new column in the calibration certificate dataset.
 - iii. will also need to convert the wavelengths of the certificate to wavenumber (use Peck and Reader 5-point conversion formula)
 - 1. save as another column in the dataset.
 - c. display a plot of the original certificate data points and the interpolated curve for a sanity check.
 - d. save the interpolated curve (in wavenumber scale) as a dataset.
- 3. Generate response curve:
 - a. $R = \text{standard lamp spectrum} / \text{interpolated calibration certificate data}$
 - b. save this response curve as new dataset.
 - c. plot the filtered standard lamp spectrum, the interpolated calibration certificate data, and the response curve for a sanity check.
- 4. Apply the response curve to the observed linelist datafile to calibrate intensities:
 - a. each wavenumber in the observed linelist needs to be matched to a datapoint in the response curve, but the observed linelist doesn't have regular spacing like the response curve.
 - i. therefore the response curve will need to be interpolated to the irregular spacing of the observed linelist.
 - ii. this then gives the R for each wavenumber in the observed linelist.
 - b. calibrate the equivalent widths (EW) in the observed linelist:
 - i. divide the original EW by the response curve value, R, for that wavenumber.
 - c. add the calibrated intensity (calibrated EW) as a column to the linelist datafile.

2. Match lines

Newly observed and previously measured lines need to be matched to the calculated wavenumbers. These will give essential information when conducting the analysis. The calculated linelist contains all possible lines from an upper level and so will be the basis of the final linelist, with previously measured (published) lines and lines from the linelists of all main spectra matched to the calculated wavenumbers.

The steps for matching are:

- 1) Match each line in each observed linelist to the calculated linelist dataset (if possible – some lines in the linelist may not match with the calculated wavenumbers):
 - a) have the user set a wavenumber matching tolerance (e.g. 0.05cm^{-1}).
 - b) match the observed wavenumber to the calculated wavenumber within the matching tolerance.
 - c) flag any lines that have multiple matches and get the user to decide identifications:
 - i) maybe suggest a change of matching tolerance if there are a lot of multiply identified lines.
 - ii) Lines might also be blended in one spectrum and clean in another. For example, Penning spectra have very few neutral lines, and lines blended with an argon line would be clean in a neon spectrum.
 - d) Add all observed line parameters to the calibrated linelist.
- 2) match the previously published lines to the calculated linelist dataset.
 - a) Same matching tolerance as for observed values.
 - b) Add previously measured data to the calculated line.
- 3) Now have a full theoretical (calculated) linelist for the spectrum with matched observed and previously published lines
 - a) when displaying this dataset to the user in the GUI, will need to make this full linelist editable to correct for any mistakes/reidentifications.
- 4) This full linelist will then have theoretical lines for all predicted transitions of interest with observed lines from each spectrum and previously published lines matched to them.
- 5)

3. Display all Information

The energy level table will need to be populated. This will contain all of the energy levels from the energy level (.lev) file. When a user selects an energy level from this table, all of the lines with that level as the upper energy level in the complete linelist dataset will be displayed by:

- Spectral plots:
 - Plots of each line in the linelist in each of the observed spectra
 - A residual plot underneath this main plot with the fitted line profile removed from the spectrum section.
 - The profile parameters are contained in the observed linelist
 - The plots will be arranged in a grid with lines of the same wavenumber in the rows and lines in the same spectrum in the columns
 - See mock GUI for details.
- Linelist:
 - All calculated lines from the upper energy level with their matched observed lines and previously measured lines
- Spectra details
 - For all spectra with lines from the upper energy level.
 - Will be able to view all files/data associated with each spectrum by clicking on them. E.g. view the response curve for a spectrum to see if there are any issues.
- Line details:
 - Comments about line, parameters in table form etc.

4. Define Common Intensity Scale

Although the observed line intensities have been calibrated, these are on a scale relative only to their own spectrum. To compare intensities between spectra, we need to define a common intensity scale.

- 1) User selects a reference line.
 - a) this should be a strong line in as many spectra as possible.
- 2) set the EW of this reference line to a large number, e.g. 1000.
- 3) scale the EW of every other line to 1000
 - a) multiply each EW by 1000/(reference line EW)
 - b) Save as a new column of data
- 4) Calculate uncertainty of lines
 - a) Calculate statistical uncertainty from equation 10 of Sikstrom et al:
 $1.5/((\text{SNR}) \cdot \sqrt{n})$
 - i) n is no. points per FWHM = FWHM / resolution
 - ii) SNR = signal-to-noise ratio
 - b) Calculate calibration uncertainty (lines farther apart have higher calibration uncertainties):
 - i) Find the strongest line from that upper level in the spectrum.
 - ii) Calculate the wavenumber separation between the line of interest and that strongest line.
 - iii) Divide the wavenumber separation by total wavenumber width of the spectrum.
 - iv) Calculate calibration uncertainty across whole spectrum from repeated measurements of the calibration and the uncertainty from the calibration certificate. This will be a user-entered variable.
 - v) Multiply the wavenumber separation by the total calibration uncertainty.
 - c) Calculate the total uncertainty as the quadrature sum of statistical and calibration uncertainty.
- 5) What happens if the user selected strong line is not in all spectra?
 - a) Select a transfer line that is present in both the spectrum without the reference line (transfer spectrum) and in the spectrum with the reference line.
 - b) Calculate the weighted average intensity for this line from all the spectra that are rescaled using the reference line.
 - i) Weighting factor is $1/(\text{unc}^2)$.
 - ii) Limit the weighting so that strong lines do not have too large an effect on the result – e.g. sets a limit so that all lines with an uncertainty of $<6\%$ ($1/(2 \cdot 0.06^2)$) get the same weight.
 - c) Multiply all lines in the transfer spectrum by the weighted average intensity of the transfer line divided by the intensity of the line in the transfer spectrum.
 - d) The uncertainty of the transfer line in the transfer spectrum is the quadrature sum of its uncertainty in that spectrum and the uncertainty of the weighted average.
 - e) The uncertainty of the other lines in the transfer spectrum is the quadrature sum of its uncertainty in that spectrum and the uncertainty of the transfer line in the transfer spectrum.
- 6) Save all uncertainties as a new column of data, noting if a transfer spectrum had to be used.

5. Exclude problem lines from analysis

Not all measured lines will be suitable for the analysis.

- 1) Calculate weighted average of scaled intensities.
 - a) Weighting factor is $1/(\text{unc}^2)$
 - b) Limit the weighting so that strong lines do not have too large an effect on the result. For example setting a limit so that all lines with an uncertainty of $<6\%$ ($1/(2 \cdot 0.06^2)$), get the same weight.
 - c) Uncertainty is $\sqrt{1/\sum(\text{weights})}$. Calibration uncertainty is included in the individual measurement uncertainties.
- 2) Calculate residuals (contributions from predicted lines not in observed spectra)
 - a) Find all the lines in the calculations involving the upper level.
 - b) The residual is the sum of all the lines that are not observed and multiply by the level lifetime.
 - c) Assign an uncertainty of 50 % to the residual.

GUI Mockup

This is a rough draft mock-up of the transition probability analysis section of the program.

